JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class AtomicIntegerArray

java.lang.Object
    java.util.concurrent.atomic.AtomicIntegerArray

All Implemented Interfaces:

Serializable

```
public class AtomicIntegerArray
extends Object
implements Serializable
```

An `int` array in which elements may be updated atomically. See the `VarHandle` specification for descriptions of the properties of atomic accesses.

Since:

1.5

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`AtomicIntegerArray(int length)`	Creates a new AtomicIntegerArray of the given length, with all elements initially zero.
`AtomicIntegerArray(int[] array)`	Creates a new AtomicIntegerArray with the same length as, and all elements copied from, the given array.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`final int`	`accumulateAndGet(int i, int x, IntBinaryOperator accumu`	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of

		applying the given function to the current and given values, returning the updated value.
final int	**addAndGet**(int i, int delta)	Atomically adds the given value to the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.
final int	**compareAndExchange**(int i, int expectedValue, int newValue)	Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.
final int	**compareAndExchangeAcquir**(int i, int expectedValue, int newValue)	Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Obje`
final int	**compareAndExchangeReleas**(int i, int expectedValue, int newValue)	Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with

		memory effects as specified by VarHandle.compareAndExchangeRelease(Obje
final boolean	**compareAndSet**(int i, int expectedValue, int newValue)	Atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.compareAndSet(Object...).
final int	**decrementAndGet**(int i)	Atomically decrements the value of the element at index i, with memory effects as specified by VarHandle.getAndAdd(Object...).
final int	**get**(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getVolatile(Object...).
final int	**getAcquire**(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getAcquire(Object...).
final int	**getAndAccumulate**(int i, int x, IntBinaryOperator accumu	Atomically updates (with memory effects as specified by VarHandle.compareAndSet(Object...)) the element at index i with the results of applying the given function to the current

		and given values, returning the previous value.
final int	**getAndAdd**(int i, int delta)	Atomically adds the given value to the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.
final int	**getAndDecrement**(int i)	Atomically decrements the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.
final int	**getAndIncrement**(int i)	Atomically increments the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.
final int	**getAndSet**(int i, int newValue)	Atomically sets the element at index `i` to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.
final int	**getAndUpdate**(int i, IntUnaryOperator updateF	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of

		applying the given function, returning the previous value.
final int	**getOpaque**(int i)	Returns the current value of the element at index i, with memory effects as specified by VarHandle.getOpaque(Object...).
final int	**getPlain**(int i)	Returns the current value of the element at index i, with memory semantics of reading as if the variable was declared non-volatile.
final int	**incrementAndGet**(int i)	Atomically increments the value of the element at index i, with memory effects as specified by VarHandle.getAndAdd(Object...).
final void	**lazySet**(int i, int newValue)	Sets the element at index i to newValue, with memory effects as specified by VarHandle.setRelease(Object...).
final int	**length**()	Returns the length of the array.
final void	**set**(int i, int newValue)	Sets the element at index i to newValue, with memory effects as specified by VarHandle.setVolatile(Object...).
final void	**setOpaque**(int i, int newValue)	Sets the element at index i to newValue, with

		memory effects as specified by `VarHandle.setOpaque(Object...)`.
`final void`	**`setPlain`**`(int i, int newValue)`	Sets the element at index `i` to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.
`final void`	**`setRelease`**`(int i, int newValue)`	Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.
`String`	**`toString`**`()`	Returns the String representation of the current values of array.
`final int`	**`updateAndGet`**`(int i, IntUnaryOperator updateF`	Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the updated value.
`final boolean`	**`weakCompareAndSet`**`(int i, int expectedValue, int newValue)`	**Deprecated.** This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(int, int, int)` and `compareAndSet(int, int, int)`).
`final boolean`	**`weakCompareAndSetAcquire`**`(int i,`	Possibly atomically sets the element at index `i` to

	int expectedValue, int newValue)	newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetAcquire(Objec
final boolean	**weakCompareAndSetPlain** (int i, int expectedValue, int newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetPlain(Object.
final boolean	**weakCompareAndSetRelease** (int i, int expectedValue, int newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSetRelease(Objec
final boolean	**weakCompareAndSetVolatil** (int i, int expectedValue, int newValue)	Possibly atomically sets the element at index i to newValue if the element's current value == expectedValue, with memory effects as specified by VarHandle.weakCompareAndSet(Object...).

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Constructor Details

AtomicIntegerArray

```
public AtomicIntegerArray(int length)
```

Creates a new AtomicIntegerArray of the given length, with all elements initially zero.

Parameters:

`length` - the length of the array

AtomicIntegerArray

```
public AtomicIntegerArray(int[] array)
```

Creates a new AtomicIntegerArray with the same length as, and all elements copied from, the given array.

Parameters:

`array` - the array to copy elements from

Throws:

`NullPointerException` - if array is null

Method Details

length

```
public final int length()
```

Returns the length of the array.

Returns:

the length of the array

get

```
public final int get(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getVolatile(Object...)`.

Parameters:

i - the index

Returns:

the current value

set

```
public final void set(int i,
                      int newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setVolatile(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

lazySet

```
public final void lazySet(int i,
                          int newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

1.6

getAndSet

```
public final int getAndSet(int i,
                           int newValue)
```

Atomically sets the element at index `i` to `newValue` and returns the old value, with memory effects as specified by `VarHandle.getAndSet(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Returns:

the previous value

compareAndSet

```
public final boolean compareAndSet(int i,
                                   int expectedValue,
                                   int newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.compareAndSet(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful. False return indicates that the actual value was not equal to the expected value.

weakCompareAndSet

```
@Deprecated(since="9")
public final boolean weakCompareAndSet(int i,
                                       int expectedValue,
                                       int newValue)
```

Deprecated.
This method has plain memory effects but the method name implies volatile memory effects (see methods such as `compareAndExchange(int, int, int)` and `compareAndSet(int, int, int)`). To avoid confusion over plain or volatile memory effects it is recommended that the method `weakCompareAndSetPlain(int, int, int)` be used instead.

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

See Also:

`weakCompareAndSetPlain(int, int, int)`

weakCompareAndSetPlain

```
public final boolean weakCompareAndSetPlain(int i,
                                            int expectedValue,
                                            int newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetPlain(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

getAndIncrement

```
public final int getAndIncrement(int i)
```

Atomically increments the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(i, 1)`.

Parameters:

`i` - the index

Returns:

the previous value

getAndDecrement

```
public final int getAndDecrement(int i)
```

Atomically decrements the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `getAndAdd(i, -1)`.

Parameters:

`i` - the index

Returns:

the previous value

getAndAdd

```
public final int getAndAdd(int i,
                           int delta)
```

Atomically adds the given value to the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`i` - the index

`delta` - the value to add

Returns:

the previous value

incrementAndGet

```
public final int incrementAndGet(int i)
```

Atomically increments the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(i, 1)`.

Parameters:

`i` - the index

Returns:

the updated value

decrementAndGet

```
public final int decrementAndGet(int i)
```

Atomically decrements the value of the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Equivalent to `addAndGet(i, -1)`.

Parameters:

`i` - the index

Returns:

the updated value

addAndGet

```
public final int addAndGet(int i,
                           int delta)
```

Atomically adds the given value to the element at index `i`, with memory effects as specified by `VarHandle.getAndAdd(Object...)`.

Parameters:

`i` - the index

`delta` - the value to add

Returns:

the updated value

getAndUpdate

```
public final int getAndUpdate(int i,
                              IntUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i` - the index

`updateFunction` - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final int updateAndGet(int i,
                              IntUnaryOperator updateFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`i` - the index

`updateFunction` - a side-effect-free function

Returns:

the updated value

Since:

1.8

getAndAccumulate

```
public final int getAndAccumulate
(int i,
 int x,
 IntBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.
The function is applied with the current value of the element at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i` - the index

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the previous value

Since:

1.8

accumulateAndGet

```
public final int accumulateAndGet
(int i,
 int x,
 IntBinaryOperator accumulatorFunction)
```

Atomically updates (with memory effects as specified by `VarHandle.compareAndSet(Object...)`) the element at index `i` with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it

may be re-applied when attempted updates fail due to contention among threads.
The function is applied with the current value of the element at index `i` as its first argument, and the given update as the second argument.

Parameters:

`i` - the index

`x` - the update value

`accumulatorFunction` - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

toString

```
public String toString()
```

Returns the String representation of the current values of array.

Overrides:

`toString` in class `Object`

Returns:

the String representation of the current values of array

getPlain

```
public final int getPlain(int i)
```

Returns the current value of the element at index `i`, with memory semantics of reading as if the variable was declared non-`volatile`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setPlain

```
public final void setPlain(int i,
                           int newValue)
```

Sets the element at index `i` to `newValue`, with memory semantics of setting as if the variable was declared non-`volatile` and non-`final`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

getOpaque

```
public final int getOpaque(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getOpaque(Object...)`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setOpaque

```
public final void setOpaque(int i,
                            int newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setOpaque(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

getAcquire

```
public final int getAcquire(int i)
```

Returns the current value of the element at index `i`, with memory effects as specified by `VarHandle.getAcquire(Object...)`.

Parameters:

`i` - the index

Returns:

the value

Since:

9

setRelease

```
public final void setRelease(int i,
                             int newValue)
```

Sets the element at index `i` to `newValue`, with memory effects as specified by `VarHandle.setRelease(Object...)`.

Parameters:

`i` - the index

`newValue` - the new value

Since:

9

compareAndExchange

```
public final int compareAndExchange(int i,
                                    int expectedValue,
                                    int newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchange(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeAcquire

```
public final int compareAndExchangeAcquire(int i,
                                           int expectedValue,
                                           int newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeAcquire(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

compareAndExchangeRelease

```
public final int compareAndExchangeRelease(int i,
                                           int expectedValue,
                                           int newValue)
```

Atomically sets the element at index `i` to `newValue` if the element's current value, referred to as the *witness value*, `==` `expectedValue`, with memory effects as specified by `VarHandle.compareAndExchangeRelease(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

the *witness value*, which will be the same as the expected value if successful

Since:

9

weakCompareAndSetVolatile

```
public final boolean weakCompareAndSetVolatile(int i,
                                               int expectedValue,
                                               int newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSet(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetAcquire

```
public final boolean weakCompareAndSetAcquire(int i,
                                              int expectedValue,
                                              int newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetAcquire(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

weakCompareAndSetRelease

```
public final boolean weakCompareAndSetRelease(int i,
                                              int expectedValue,
                                              int newValue)
```

Possibly atomically sets the element at index `i` to `newValue` if the element's current value == `expectedValue`, with memory effects as specified by `VarHandle.weakCompareAndSetRelease(Object...)`.

Parameters:

`i` - the index

`expectedValue` - the expected value

`newValue` - the new value

Returns:

`true` if successful

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.